



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Алекса Попов

?????

ДИПЛОМСКИ РАД
- Основне академске студије -

Нови Сад, 2021



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски (Bachelor) рад
Аутор, АУ:	Алекса Попов
Ментор, МН:	Др Немања Недић
Наслов рада, НР:	???
Језик публикације, ЈП:	Српски / ћирилица
Језик извода, ЈИ:	Српски
Земља публиковања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2021
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад; трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	9/26/0/0/30/0/0
Научна област, НО:	Електротехника и рачунарство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Интеграција електроенергетских система
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	
Датум прихватања теме, ДП:	
Датум одбране, ДО:	
Чланови комисије, КО:	Председник: др ???
	Члан: др ???
	Члан, ментор: др ???
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	Bachelor Thesis
Author, AU :	Aleksa Popov
Mentor, MN :	Nemanja Nedić, Ph. D
Title, TI :	A Software Tool for Animal Shelter Support Service
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2021
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	9/26/0/0/30/0/0
Scientific field, SF :	Electrical and computer engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Integration of power systems
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad, Serbia
Note, N :	
Abstract, AB :	
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	
Defended Board, DB :	President: ??, Ph. D
	Member: ??, Ph. D
	Member, Mentor: ??, Ph.D
	Menthor's sign

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Датум:
	ЗАДАТАК ЗА ДИПЛОМСКИ РАД	Лист/Листова:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Примењено софтверско инжењерство
Руководилац стидијског програма:	???

Студент:	Алекса Попов	Број индекса:	ПР 113/2017
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	др Немања Недић		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА МАСТЕР РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ДИПЛОМСКОГ РАДА:

???

ТЕКСТ ЗАДАТКА:

--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора

Садржај

1. Увод	1
2. Опис проблема	4
3. Теоријско-технолошке основе.....	4
3.1 Вишеслојне апликације	4
3.2 Развој веб апликације.....	5
3.3 Web API	6
3.4 Коришћене технологије	7
3.5 Зашто React?	9
4. Модел података	10
5. Backend функционалност	12
6. Архитектура frontend апликације	13
6.1 Сталне компоненте	14
6.2 Компоненте променљивог приказа	14
7. Кориснички интерфејс	15
7.1 Распоред компоненти	15
7.2 Navbar	16
7.3 Sidebar	17
7.4 Dashboard.....	18
7.5 Substation приказ	20
7.6 Приказ листе појединачних прекидача	21
7.7 Промена величине прозора и сајдбара	23
8. Закључак	25
9. Литература.....	26

1. Увод

Електроенергетски систем представља сложени, динамички систем, чија је функција првенствено да сигурно, поуздано, квалитетно и економично снабдева потрошаче довољним количинама квалитетне електричне енергије. У последње време потражња за оваквим системима је у великом расту, а самим тим и ниво захтеваног квалитета је висок. Да би се обезбедио критеријум квалитета који се захтева потребно је имати добар и поуздан софтвер. Он моделује и анализира системе за дистрибуцију електричне енергије са свим детаљима од трансформаторске станице до самих корисника. Добро дизајниран електроенергетски систем осигурава добре перформансе и доводи расположивост постројења до највише тачке у свим радним условима, укључујући пролазне услове попут нелинеарних оптерећења и губитака енергије.

Common Information Model (CIM) је развијен и одржаван ради пружања заједничке дефиниције управљачких информација за систем, мреже и услуге и дозвољава проширења добављача. CIM стандарди се континуално развијају како би задовољили променљиве захтеве за размену података, који се повећавају и по учесталости и по врсти, са већом RES интеграцијом и увођењем паметних мрежа. CIM шема пружа стварне описе модела. Шеме управљања су градивни елементи за команде платформе, као што су конфигурација уређаја, управљање променама и перформансама. CIM структура описује међусобно повезане системе, сваки састављен од дискретних елемената. Снабдевајући скуп класа својствима и асоцијацијама које пружају разумљив оквир, CIM организује информације о окружењу којим управљамо. Стандард који дефинише основне пакете CIM-а је IEC 61970-301. Фокусира се на потребе преноса електричне енергије као и на главне компоненте које обухватају информациони систем, SCADA, планирање, оптимизацију итд.

Веб платформе посредују у протоку информација на интернету. Оне су такође кључни покретачи дигиталне трговине на јединственом тржишту широм света. Повећавају избор и погодност потрошача, ефикасност и конкурентност индустрије и могу повећати учешће грађана у разним активностима. Све је већи број информација које је потребно пренети, као и сам број корисника које имамо у систему. Предност веб апликације представља њена способност да брзо пренесе и учитава информације. Следећа предност, у односу на десктоп апликације, је да пружа широк спектар пословних предности. Овим апликација се може приступити са било ког рачунара путем интернета, уместо да се појединачно инсталирају на сваком рачунару са ког се жели приступити. Уз апликације засноване на вебу, сви корисници приступају систему путем јединственог окружења веб претраживача.

Путем самог сајта, посетилац који води евиденцију о реалном систему, може брзо и ефикасно проверити бројно стање, статус и остале информације о опреми. Такође убрзава процес штампања стања, налажење жељене, појединачне опреме као и унос, брисање и измена постојеће. Путем разних типова евиденција веб сајт може да има ограничен број корисника који воде рачуна о датој опреми.

Због горе наведених примера веб сервис представља једано од најбољих решења за приказ и реализацију основних функционалности једног реалног система.

2. Опис проблема

Реализовати апликацију за надгледање стања и извршавање измена над електроенергетским системом. Акценат је на једноставности употребе и прилагодљивости апликације за рад на датом систему.

Систем се састоји од трансформаторских станица које поседују прекидаче. Прекидачи су подељени у 4 врсте:

- Disconnector
- Fuse
- LoadBreakSwitch
- Breaker

Прва страница коју корисник види је приказ броја свих елемената система и бројно стање у свакој трансформаторској станици. Дате информације потребно је приказати и у графичком облику. На почетној страни налази се и „to-do“ листа која омогућава додавање, брисање и чекирање разних догађаја. Корисник има понуђену листу свих трансформаторских станица на које може приступити након чијег приступа му се пружа могућност да уђе на преглед појединачних типова прекидача те трансформаторске станице. На страници станице могуће је видети све податке о њој и изменити их по жељи. Приказ свих појединачних прекидача даје могућност корисницима да мењају или бришу појединачне прекидаче.

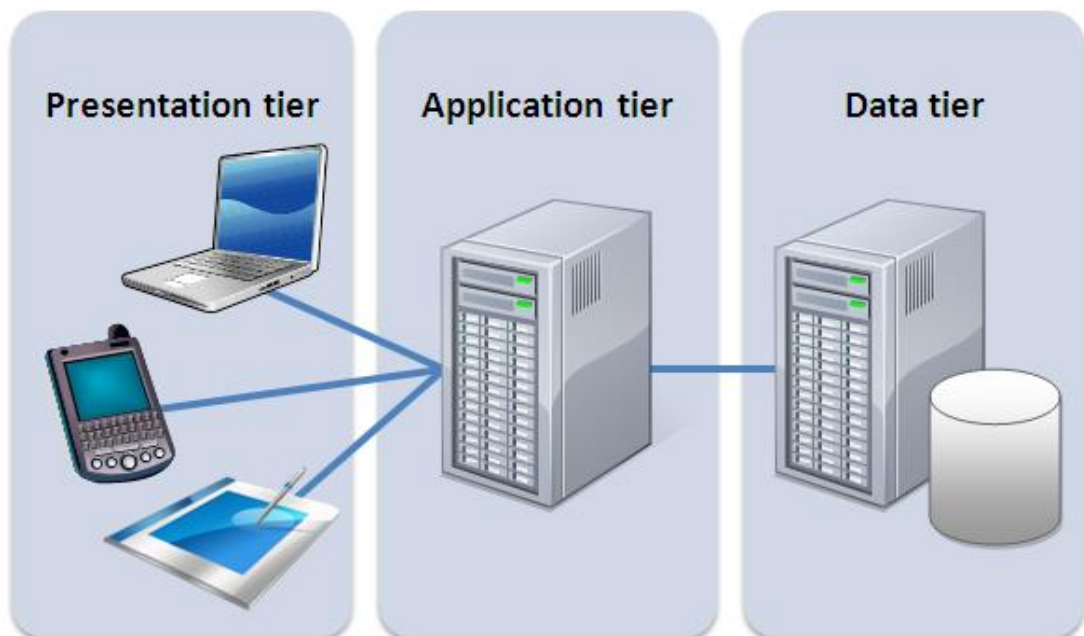
Сваки прекидач као и сама трансформаторска станица има своју јединствени симбол. Могућа је опција мењања симбола у зависности од тржишта за који је сајт коришћен (Европско или Америчко).

3. Теоријско-технолошке основе

3.1 Вишеслојне апликације

У софтверском инжењерингу, вишеслојна архитектура (често споменућа као “n-tier” архитектура) је client-server архитектура у којој су функције презентација, обраде апликација и управљања подацима физички одвојене. Трослојна архитектура представља најпопуларнију вишеслојну архитектуру. Трослојна архитектура је добро успостављена архитектура софтверских апликација која организује апликације у три логичке и физичке слојеве рачунања:

1. Ниво презентације или кориснички интерфејс
2. Ниво апликације
3. Ниво података



Слика 1 – Трослојна архитектура

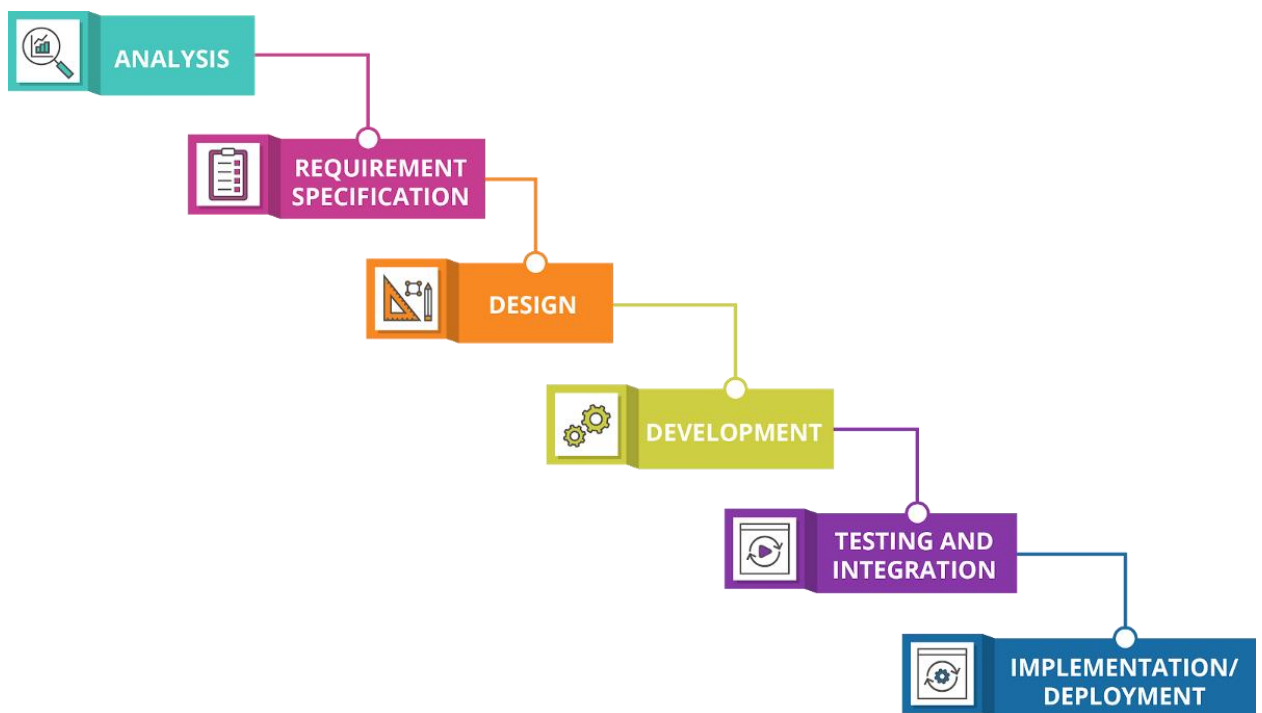
Презентациони слој је кориснички интерфејс и комуникацијски слој апликације, где крајњи корисник ступа у интеракцију са апликацијом. Његова главна сврха је приказивање информација и прикупљање података од корисника. Овај слој се најчешће реализује са графичким корисничким интерфејсом (GUI), а може да ради на веб претраживачу, као десктоп апликација, или као мобилна апликација. Презентациони слој веб апликације обично се развија коришћењем HTML-а, CSS-а и JavaScript-а. Десктоп апликације се могу писати на различитим језицима у зависности од платформе. Апликативни слој, познат као логички или средњи слој, језгро је апликације. На апликативном слоју се обрађују информације прикљене са презентационог слоја. Понекад и у односу на друге информације на нивоу података користећи пословну логику, која представља одређени скуп правила. Апликативни слој такође може да додаје, брише или мења податке у слоју података.

Слој података је место где се складиште информације и управља њима. Ово може бити систем базе података као PostgreSQL, MySQL, MariaDB, Oracle, DB2, Informix или у NoSQL базама података као што су Cassandra, CouchDB или MongoDB.

3.2 Развој Веб апликације

Планирање и развој веб апликације игра велику улогу у завршном квалитету саме апликације. Овај корак је неизоставан како би се обезбедило максимално задовољство клијента. Такође, недовољно планиран приступ развоју често доводи до веће цене развоја апликације, чиме се ризикује да се апликација никада не развије до краја услед високих непланираних трошкова. Развој веб апликације је развој у различитим фазама које садрже активности са намером за боље планирање и управљање. Постоји доста развојних метода апликација које се користе већ дужи низ година као што су модел водопад, спирални модел, итеративни модел, еволицијски модел, итд.

Модел водопад настао је у раним 70-тим годинама 20. века, као непосредна аналогија са процесима из других инжењерских струка. Софтверски процес је грађен као низ временски одвојених активности, у складу са сликом.



Слика 2 – Модел Водопад

Модел је добио име због облика дијаграма који подсећа на водопад. Активности се одвијају као фазе секвенцијално једна иза друге. Свака фаза даје неки резултат који „тече“ по моделу и представља полазиште за наредну фазу. Овај модел омогућава детаљно планирање целог софтверског процеса и поделу послова. Лако се може утврдити стање у којем се процес тренутно налази.

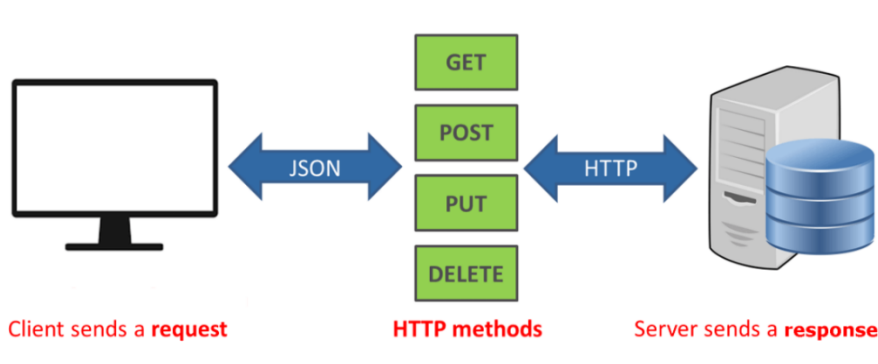
Највећа мана модела водопад је што не одржава стварни начин на који се код развија, јер се софтвер обично развија кроз велики број итерација. Нестандардне технике развоја софтвера постоје због различитих проблема из реалног света који описују.

3.3 Web API

API (Application Programming Interface) је интерфејс за програмирање који дефинише начине на које апликације могу да захтевају услуге од библиотека или оперативних система. Такође омогућава да два одвојена софтверска програма међусобно комуницирају. А RESTful API или REST API заснован је на REST-у (representational state transfer). REST API представља архитектонски стил и приступ комуникацији који се често користи у развоју веб услуга. REST технологија се генерално преферира у односу на друге сличне технологије, зато што REST користи мању пропусност, што га чини погоднијим за ефикасно коришћење интернета. RESTful API се такође може изградити помоћу програмских језика као што су JavaScript или Python.

RESTful API користи команде за добијање ресурса. Стање ресурса у било којој датој временској ознаци назива се „resource representation“. Он користи постојеће HTTP методологије дефинисане протоколом RFC 2616, као што су:

- GET – за преузимање ресурса
- PUT – за мењање стања или ажурирање ресурса, који може бити објекат, датотека или блок
- POST – за стварање новог ресурса
- DELETE – за његово брисање



Слика 3 - REST API

ASP.NET Web API је прошириво окружење за изградњу услуга заснованих на HTTP-у којима се може приступити у различитим апликацијама на различитим платформама, попут веба, мобилних уређаја, итд. ASP.NET Web API ради слично као и веб апликација писана у ASP.NET MVC. Једина разлика је у томе што ASP.NET Web API шаље податке као одговор уместо HTML приказа, што је стандардно за ASP.NET MVC. Сви прегледачи имају скуп уграђених Web API-а који подржавају сложене операције и помажу у приступу подацима. На пример, Geolocation API може да додави координате одређених физичких места.

Web API	WCF
Open source and ships with .NET framework.	Ships with .NET framework
Supports only HTTP protocol.	Supports HTTP, TCP, UDP and custom transport protocol.
Maps http verbs to methods	Uses attributes based programming model.
Uses routing and controller concept similar to ASP.NET MVC.	Uses Service, Operation and Data contracts.
Does not support Reliable Messaging and transaction.	Supports Reliable Messaging and Transactions.
Web API can be configured using HttpConfiguration class but not in web.config.	Uses web.config and attributes to configure a service.
Ideal for building RESTful services.	Supports RESTful services but with limitations.

Слика 4 – Главне разлике између Web API-а и WCF-а

3.4 Коришћене технологије

У наставку су описане главне технологије и алати коришћени у изради пројекта, као и разлози за њихов одабир.

- **JavaScript** – JavaScript је програмски језик заснован на тексту који се користи и на страни клијента и на серверу и омогућава корисницима да веб странице буду интерактивне. Тамо где HTML и CSS су језици који дају структуру и стил веб страницама, JavaScript даје веб страницама интеркативне елементе. Уобичајни примери JavaScript-а које људи свакодневно користе су на пример, оквир за претрагу на Amazon-у, видео запис са вестима уграђен на The New York Times сајту, или освежавање вашег Твитер канала. Коришћењем JavaScript-а се корисничко искуство знатно побољшава, пребацивањем из статичке у интерактивну веб страницу.
- **React.js** - React.js је „open-source“ JavaScript библиотека која се користи за изградњу корисничког интерфејса посебно за апликације са једном страницом. Користи се за руковање слојем приказа за веб и мобилне апликације. React нам такође омогућава да креирамо компоненте корисничког интергејса за виšekратну употребу. Он омогућава програмерима да креирају белике веб апликације које могу мењати податке, без поновног учитавања странице. Главна сврха React-а је да буде брз, скалабилан и једноставан. Ради само на корисничким интерфејсима у апликацији. Ово одговара приказу MVC (Model-view-controller) архитектуре. Може се користити у комбинацији са другим JavaScript библиотекама или окружењима, као што су Angular JS у MVC-у.
- **Redux** – Redux је место за складиштење стања променљивих у апликацији. Он креира процес и процедуре за интеракцију са складиштем тако да компоненте неће насумично читати нити ажурирати складиштене елементе. Укратко Redux је начин управљања стањем или се може рећи да је то складиште којем све компоненте могу приступити на структуриран начин. Мора се приступити путем Reducer-а и Action-а.
- **Bootstrap** – Bootstrap је најпопуларнији HTML, CSS и JavaScript окружење за развој прилагодљиве веб странице, такође прилагођене за мобилне уређаје. Представља front-end оквир за лакши и бржи веб развој. Садржи HTML и CSS засноване дизајн оквире за форме, дугмиће, табеле, навигације, ротирајуће слике и мнгове друге елементе. Може користити разне JavaScript датотеке. Омогућава програмеру да креира одговарајући интерактивни дизајн.
- **HTML** – HTML (Hypertext Markup Language) је језик за означавање који дефинише структуру веб садржаја. Састоји се од низа елемената који се користе за затварање или премотавање различитих делова садржаја како би га приказао на одређени начин или деловали на одређени начин. HTML ознаке могу направити реч или слику са hyperlink-ом ка некој другој локацији, могу направити фонт мањим или већим, итд.

- **CSS** – CSS је језик за описивање презентације веб страница, изглед и фонтове. Омогућава прилагођавање презентације различитим врстама уређаја, попут великих екрана, малих екрана или штампача. Он је независан од HTML-а и може се користити са било којим „XML-based markup“ језиком. Одвајање HTML-а од CSS-а олакшава одржавање сајта, дељење стилских листова по страницама и прилагођавање страница различитим окружењима.
- **Visual Studio Code** – Visual Studio Code је уређивач изворног кода који је развио Microsoft за Windows, Linux и Mac OS. Подржава отклањање делова синтаксе, интелигентно довршавање кода, одломке и рекатсорисање кода. Веома је прилагодљив, омогућава корисницима да мењају тему, пречице на тастатури, подешавања и инсталирају прикључке за додатне функционалности. Уместо система пројеката, VS Code омогућава кориснику да отвори један или више директоријума, што затим може бити сачувано као радни простор за поновну употребу. Ово омогућава да ради као језичко-агностички уређивач кода за било који језик.

3.5 Зашто React?

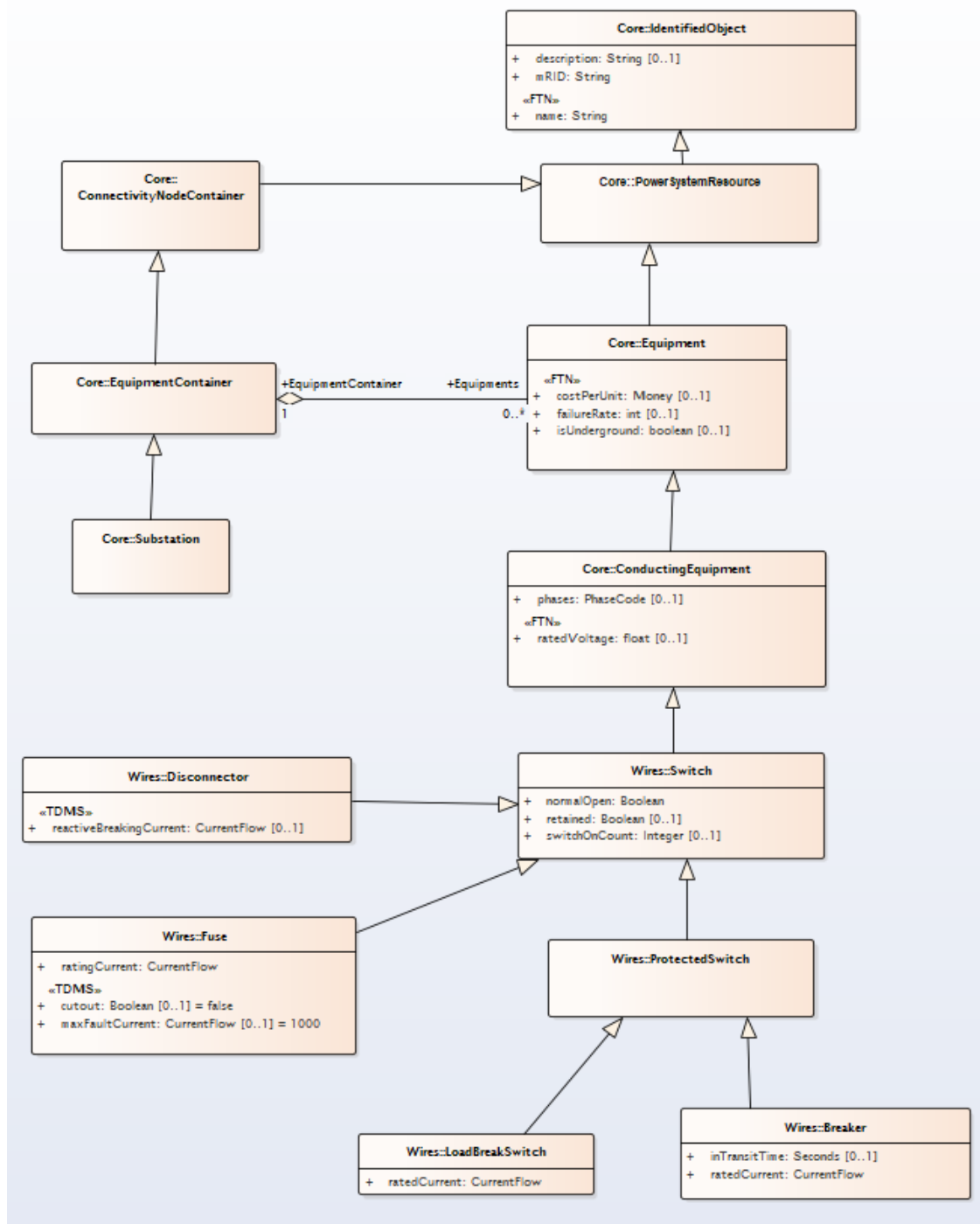
У области frontend програмирања постоји велики избор библиотека и радних оквира, као што су Vanilla, React, Angular, View, Svelte, Lit и многи други

Главни разлози за употребу React.js и његове предности над конкуренцијом су:

1. Једноставан за употребу и учење – Схватање основа убрзо након упознавања са оквиром омогућава програмерима да брже улазе у пројекат, чинећи одређени софтвер, радни оквир или библиотеку не само добрим, већ и економичним алатом, што је велики плус.
2. Изградња података – Функција једносмерне изградње података и цела архитектура (која се назива и Flux) стварају ефикасан проток података до компоненти преко диспечера – једноставне контролне тачке. Овом функцијом је уклањање грешака великих апликација много лакше.
3. Перформансе и тестирање – Постоје велике предности виртуалног DOM-а (Document Object Model). Ова особина може значајно побољшати перформансе веб сајта, посебно код већих, динамичних корисничких интерфејса. Са друге стране React, користи Webpack да би разврстао зависности, а на располагању је широк спектар различитих модула који омогућавају да се баве различитим пројектима и ситуацијама када је Webpack одсутан, а постоје зависности за сређивање.
4. Декларативност – код писан у React-у се више фокусира на оно што се приказује него на стварне кораке који до њега воде. Ова метода пре свега значи побољшано искуство за програмере што се касније претвара и у боље корисничко искуство.
5. Компонентни приступ – Апликације се граде у React-у изградњом блокова – компоненте одфоровне за UI функције, мрежне комуникације и још сложеније функције попут управљања стањима. Овај специфичан приступ чини имплементацију дизајнерских система једноставном и поједностављеном за програмере.
6. Минимализам – React нуди минималистички приступ, заузима мало простора и лако се преузима. Конфигурација је такође једноставна, а функција за поделу кода чини га једноставним када је у питању корисничко искуство, нарочито код већих пројеката.
7. Фелксибилност – Када се клијенти саветују са програмерима, они обично желе веб сајт који је једноставан за употребу и трајан. Због тога се најчешће користе технологије који су већ популарни са мноштвом опција. Поседовањем прошириве библиотеке и могућности за даљи развој, омогућено је олакшано прилагођавање током рада.
8. Компатибилност са старијим верзијама – Ово је такође пресудна карактеристика када је у питању избор одређене технологије. Могућност ажурирања или рада са старијим верзијама софтвера може уштедети много времена и новчаних средстава. Не постоји много окружења или језика који нуде потпуну компатибилност уназад, за разлику од React-а који нуди ову могућност.

4. Модел Података

Модел података, чији је UML дијаграм креиран помоћу Enterprise Architect-a, служи као оријентир за моделовање објеката и интеракција између њих. Он је дефинисан CIM профилем. Приказани модел система је упрошћен у великој мери. Међутим сама комплексност овог модела не игра главну улогу у комплексности решавања самог проблема.



Слика 5. – Модел Података

Добро дефинисано решење једноставног модела омогућује једноставну имплементацију и у случају повећања комплексности модела

Модел података састављен је од тринаест класа од којих су осам апстрактне а пет су конкретне. Класе на које ће се овај рад фокусирати, и које су од главног значаја јесу конкретне класе јер корисник на самом приказу сајта не мора да зна, нити ће му бити приказана логика целог система.

Пет главних конкретних класа чине:

- **Substation** – Трансформаторска станица
- **Disconnecter** – Уређај за брзо искључивање електронских система са њиховог примарног извора напајања
- **Fuse** – Прекидач који се користи за одвајање електричне опреме од електричне мреже
- **LoadBreakSwitch** - Прекидач за искључивање дизајниран да обезбеди стварање или прекид одређених струја
- **Breaker** - Прекидач који је дизајниран да обезбеди укључивање и искључивање у систем.

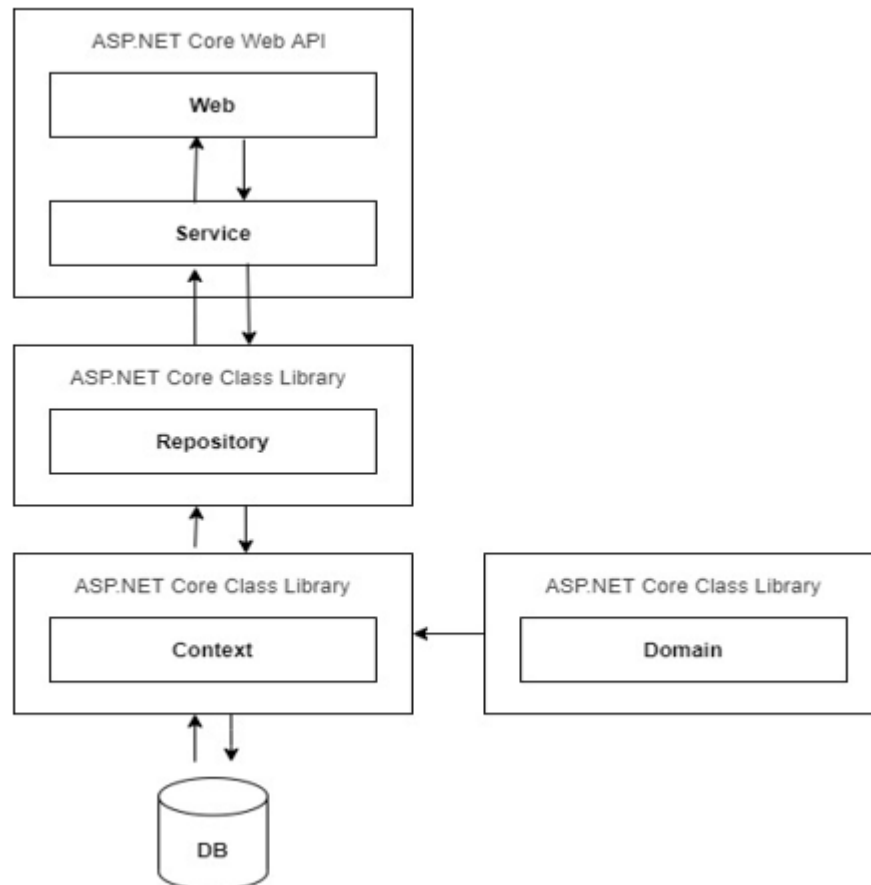
Ентитети Disconnecter, Fuse, LoadBreakSwitch и Breaker су деца ентитета Switch (прекидач) . Они су груписани као листа прекидача у трансформаторској станици. Ови ентитети ће заједно са станицом чинити суштину овог рада и приказ ће се фокусирати на њих. Поред наведених атрибута самих ентитета додати су, ради функционалности самог рада и следећи атрибути:

- Id – јединствени идентификатор
- CreateDateTime – време креирања ентитета
- LastModifiedDateTime – време брисања ентитета

Напоменуте промене су круцијалне за основне операције које се врше на пројектном раду. Јединствени идентификатор служи за праћење података и њихово нумерисање. Омогућује да брисање и модификација ентитета увек буду извршене над траженим објектом. Време креирања и брисања ентитета, поред одређених значаја за задњу страну, на предњој страни служе за праћење бројног стања и измена чија сама употреба ће бити детаљније објашњена у даљем наставку рада.

5. Backend функционалност

Пројектни рад, као што је већ споменуто, ослања се на већ израђену задњу страну (backend). У следећем пасусу биће укратко објашњена софтверска структура те стране.



Slika 6 – Архитектура решења

На слици је приказана структура наше веб апликације. Искоришћена је већ објашњена, вишеслојна архитектура софтвера. Подељена је у слојеве ради олакшаног рада над сложеном апликацијом. Компоненте унутар одређеног слоја комуницирају само са суседним слојевима помоћу добро дефинисаних интерфејса (Dependency injection). Све компоненте у заједници обезбеђују главну функционалност, док се свака компонента бави логиком која се само односи на тај слој. Презентациони слој бави се приказом података корисницима и биће детаљније објашњен у наставку рада. Сервисни слој комуницира са слојем за приступ бази који је одговоран за извршавање потрених упита над базом. База се креира на основу Domain dll-a, који садржи модел података, и Context dll-a који користи тај модел података за креирање миграција и омогућава приступ бази података слоју за приступ подацима.

6. Архитектура frontend апликације

Предња страна апликације је подељена у више компоненти. Компоненте су подељене на два типа, оне које су стално приказане и оне које се мењају у зависности од корисничке акције. Све компоненте су прављене узимајући у обзир разне димензије екрана. С циљем да корисник има једнако добро видљив приказ и на екрану мобилног телефона, али и на великом екрану персоналног рачунара. Свака компонента се састоји из одређених стања (state), за извршавање одређених акција које су јединствене за њу. Такође, свака компонента користи хукове (React Hooks) за преузимање стања из Redux складишта. Redux ажурира стања апликације користећи Actions, што представља добављање података из базе, користећи пакет Axios.

Хукови (React Hooks) омогућавају коришћење стања и других ствари у React-у, без писања засебних класа. Такође, на врло моћан и експресиван начин омогућују поново искоришћавање функциоаности између компоненти.

Redux складиште представља једноставан и практичан начин за чување стања унутар апликације.

Axios је једноставан HTTP клијент за веб прегледник заснован на систему „обећања“ (promise based). Пружа веома компактну и једноставну библиотеку са проширивим интерфејсом.

Поред овога, апликација је писана на начин да неке ствари добавља из локалног складишта (local storage), користећи пакет Redux Persist. Пример функционалности која је остварена на овај начин је „to-do“ листа.

Redux Persist је библиотека која омогућава чување Redux store-а у локалном складишту апликације.

The screenshot shows a code editor with two panels. The left panel displays the contents of a file named `store.js`. The code imports `createStore`, `applyMiddleware` from 'redux', `reducers` from './reducers/index', `composeWithDevTools` from 'redux-devtools-extension', `thunkMiddleware` from 'redux-thunk', and `persistStore` from 'redux-persist'. It then creates a `store` using `createStore` and `composeWithDevTools`, wraps it with `persistStore` to create a `persistor`, and exports both. The right panel shows a file explorer for a folder named 'Redux'. It contains subfolders for `actionCreators`, `actions`, `constants`, and `reducers`, along with a `store.js` file.

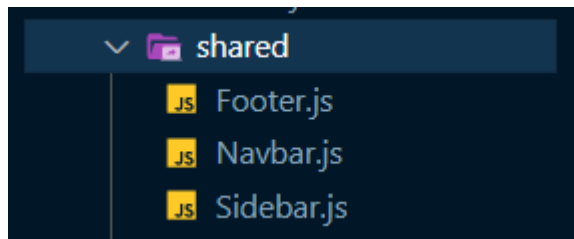
```
src > app > Redux > store.js > ...
1  import {createStore, applyMiddleware} from 'redux';
2  import reducers from './reducers/index';
3  import {composeWithDevTools} from 'redux-devtools-extension';
4  import thunkMiddleware from 'redux-thunk';
5  import {persistStore} from 'redux-persist';
6
7  export const store = createStore(reducers,composeWithDevTools( applyMiddleware(thunkMiddleware) ));
8
9  export const persistor = persistStore(store);
10
11 export default {store, persistor};
12
13 import {PersistGate} from 'redux-persist/integration/react'
14
15 ReactDOM.render(
16   <BrowserRouter basename="/demo/corona-react-free/template/demo_1/preview">
17     <Provider store={store}>
18       <PersistGate persistor={persistor}>
19         <App />
20       </PersistGate>
21     </Provider>
22   </BrowserRouter>
23   , document.getElementById('root'));
24
25 serviceWorker.unregister();
```

Слика 7 – Имплементација Redux Persist-а и структура Redux-а

6.1 Сталне компоненте

Сталне компоненте су компоненте које су стално видљиве кориснику. Најчешће су то разне опције за навигацију по веб сајту, наслов, хедер (header), футер (footer). У апликацији тренутно постоје сајдбар (sidebar, навигациони мени са леве стране), навбар (navbar, мени са опцијама на горњој страни), као и стандардни футер.

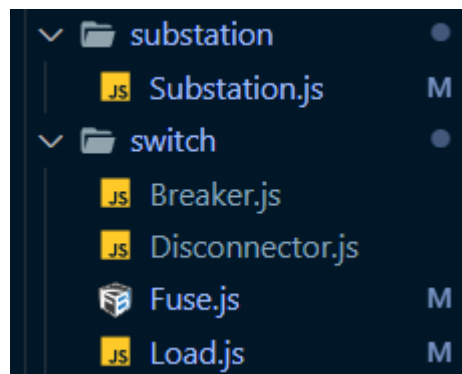
Сајдбар садржи елементе (трафостанице) које добавља из базе података, док навбар има опције одабира за које тржиште ће бити рађен приказ (Европско и Америчко тржиште имају различите ознаке у приказу) као и могућност додавања нових трафостаница.



Слика 8 – Сталне компоненте

6.2 Компоненте променљивог приказа

Компоненте променљивог приказа су све оне компоненте чији приказ на корисничком интерфејсу зависи директно од акције корисника. На пример, одабиром неке трафостанице приказују се све информације о њој, али и основне информације о елементима унутар трафостанице. Такође, могућ је и детаљан приказ информација о сваком елементу трафостанице. Додатно, кориснику је омогућено да на врло једноставан начин има и графички приказ о бројном стању елемената, који се освежава у реалном времену.



Слика 9 – Компоненте променљивог приказа

7. Кориснички интерфејс

У овом поглављу ће бити детаљно објашњен начин коришћења описане апликације. Приказ управљања апликацијом дефинисаће пројекат веома погодан за кориснике и лак за употребу, што представља и главни фокус овог задатка.

7.1 Распоред компоненти

Као што је већ описано у прошлом поглављу, компоненте се деле на сталне и компоненте променљивог приказа.

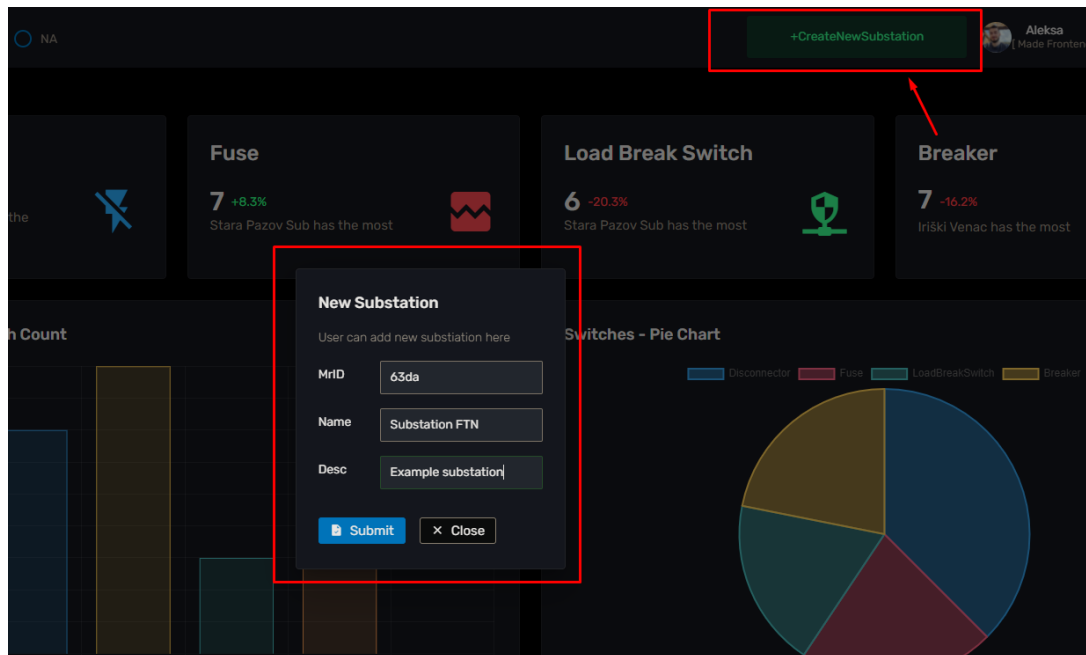


Слика 10 – Почетна страна

На слици, црвеним оквирима, одвојене су дате компоненте. (Слика 10) У горњем делу налази се навбар који садржи пар функционалности. То су додавање нове трансформаторске станице, мењање тржишта за који се користи сајт (измена иконица) и проширење и сужавање сајдбара. Сам сајдбар налази се у левом делу апликације. Њега чине путање ка осталим страницама које чине трансформаторске станице и приказ листа прекидача за сваку станицу. У самом центру апликације налази се прозор са приказом активне странице. У моменту покретања апликације то је dashboard страница. Детаљи рада сваке од наведених компоненти ће бити објашњени у наставку.

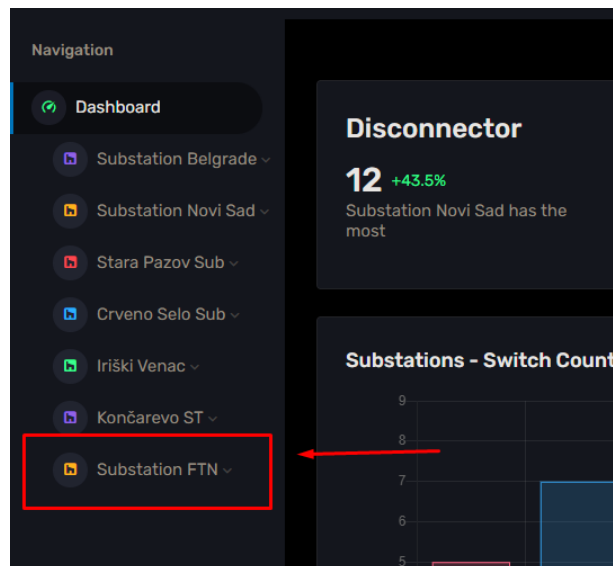
7.2 Navbar

Навбар или навигациони прозор има три основне функционалности. Прва функционалност је додавање нове трансформаторске станице. Кликом на дугме „+CreateNewSubstation“ кориснику се отвара прозор за попуњавање потребних атрибута за дату класу. (Слика 11)



Слика 11 – Додавање нове станице

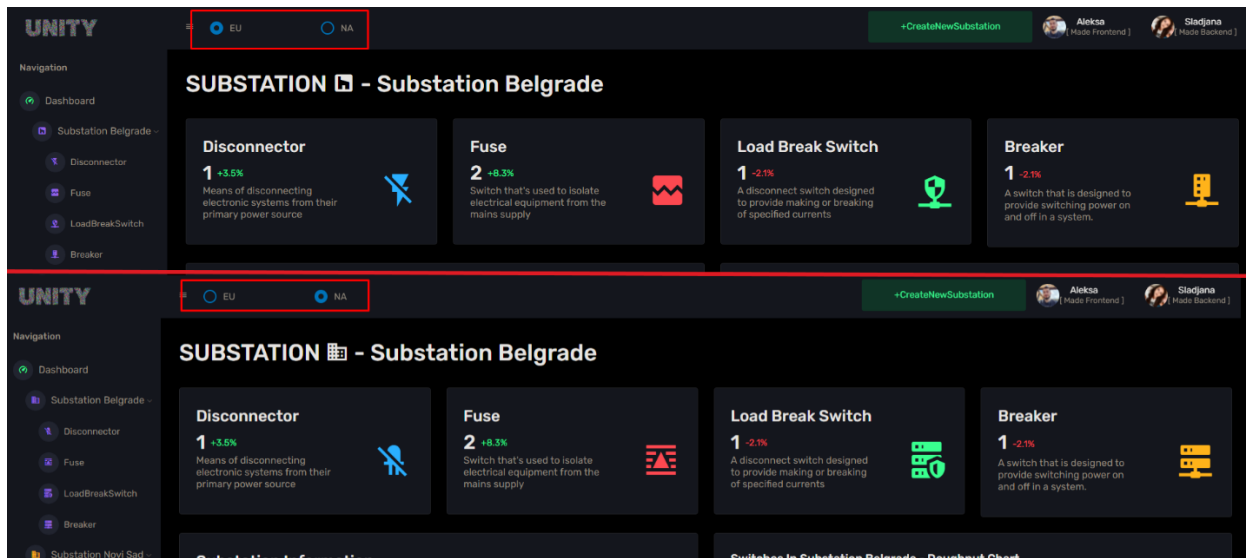
Потребно је навести да приказана форма за унос новог елемента, као и свака друга форма у овој апликацији имају одређен ниво валидације која спречава унос погрешних података. Након уноса нове станице, на сајдбару ће бити додата нова компонента којој се може приступити. (Слика 12)



Слика 12 – Приказ новонастале станице на сајдбару

Друга функционалност навбара јесте промена тржишта за које се сајт користи. Сваки прекидач као и трансформаторска станица имају јединствену иконицу која представља симбол овог ентитета. При промени тржишта и сама иконица се мења у иконицу која одговара изабраном тржишту.

Checkbox на левој страни навбара омогућава ову радњу (Слика 13), једноставним кликом на пољем са именом жељеног тржишта.

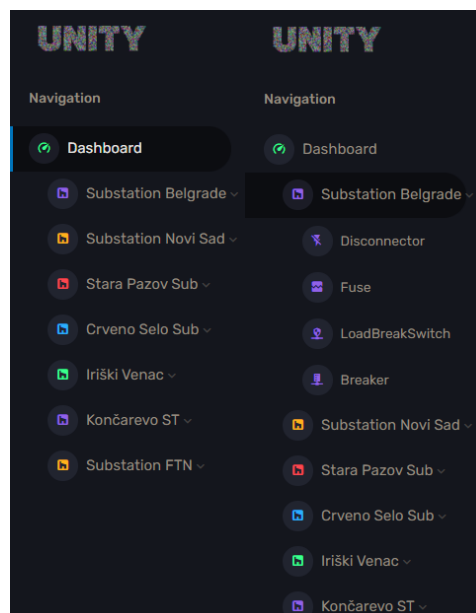


Слика 13 – Разлика Евроског и Америчког тржишта на апликацији

Трећа функционалност јесте промена величине сајдбара. Ова радња ће бити детаљно објашњена у поглављу „Промена величине прозора и сајдбара“.

7.3 Sidebar

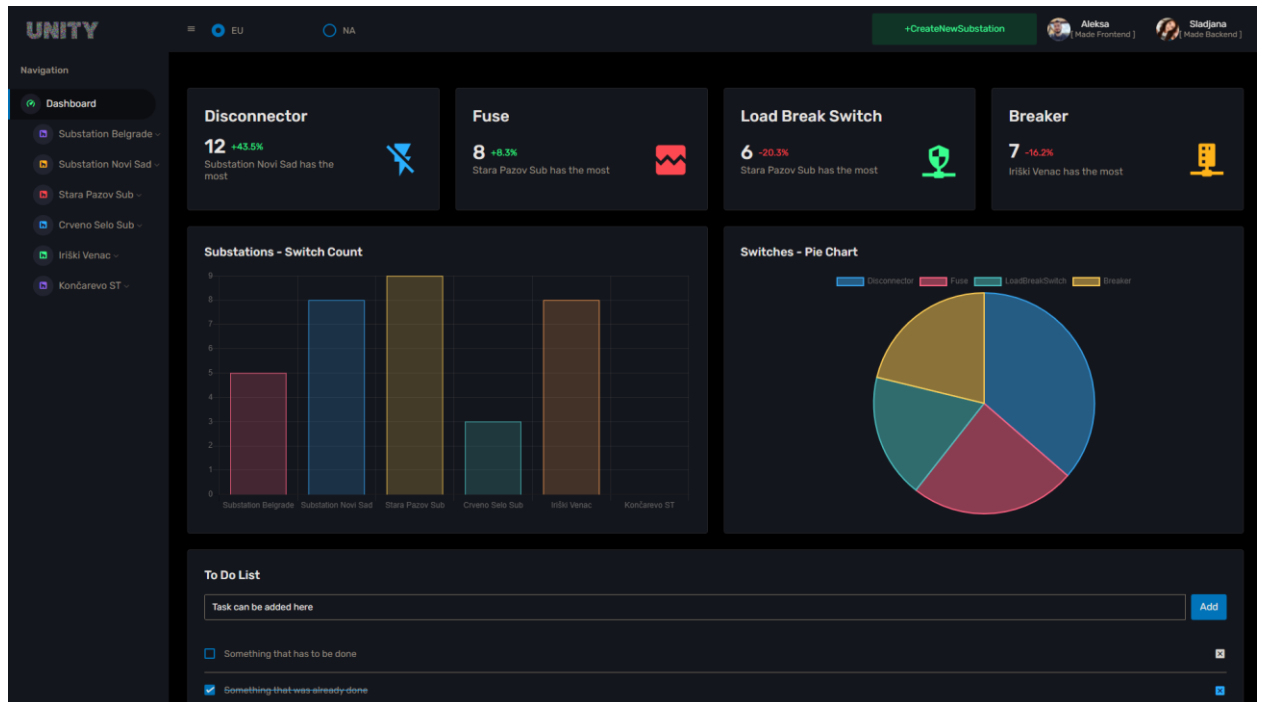
Сајдбар нам служи да приступимо осталим страницама ове апликације. На њему се налази мени који нам омогућава приступ Dashboard-у, као и свим трансформаторским станицама у апликацији. Кликот на неку од трансформаторских станица, апликација нам приказује детаљније информације о тој станици и приказује падајући мени испод кликнуте станице. (Слика 14) Падајући мени састоји се од 4 елемента. Преглед листа за сваки од 4 прекидача за станицу која је изабрана. Кликот на Dashboard корисник се враћа на почетну страницу и тренутно активни падајући мени се затвара.



Слика 14 – Функционалност сајдбара

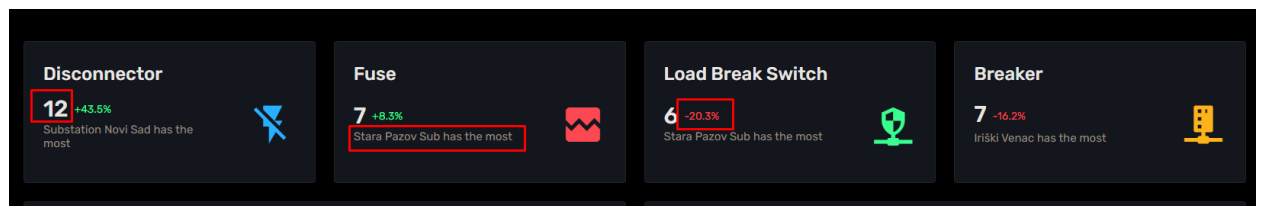
7.4 Dashboard

Dashboard део је почетне странице сајта. На њему се налазе основне информације о самом систему. Састоји се од три основна елемента. (Слика 15)



Слика 15 – Dashboard страница

Први елемент Dashboard-а јесу блокови са информацијама о сваком типу прекидача. Они садрже бројне вредности прекидача у целом систему, информацију која трансформаторска станица има највећи број датог прекидача у систему и колико (по проценту) се број прекидача повећао у односу на прошлу недељу. (Слика 16)

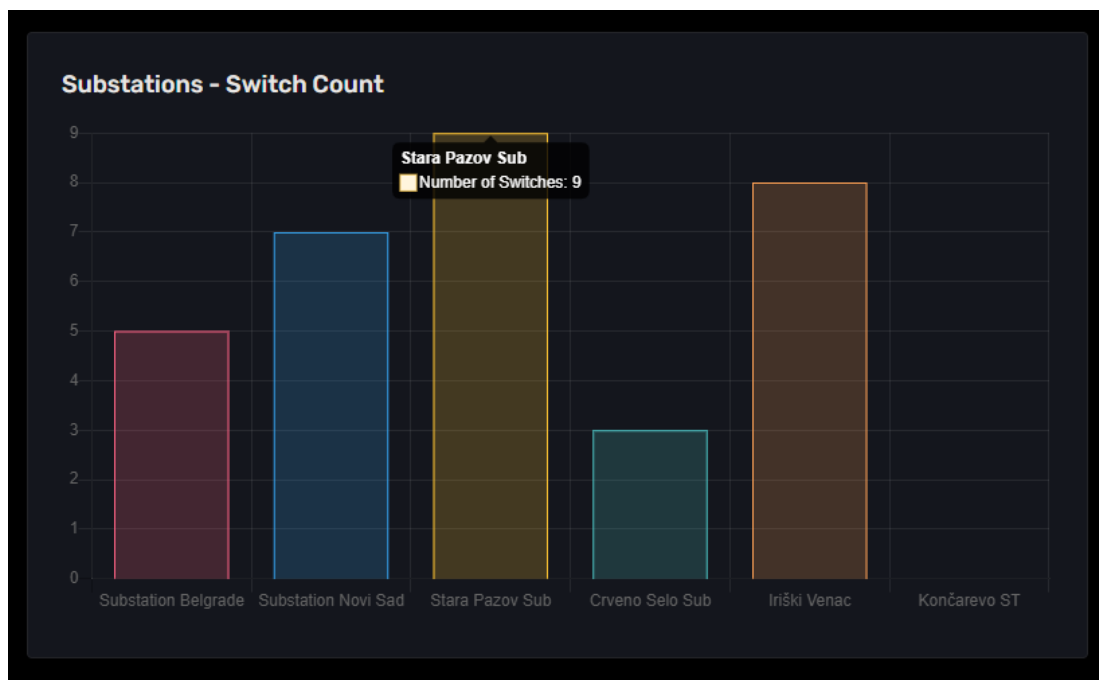


Слика 16 – Блокови са информацијама о прекидачима

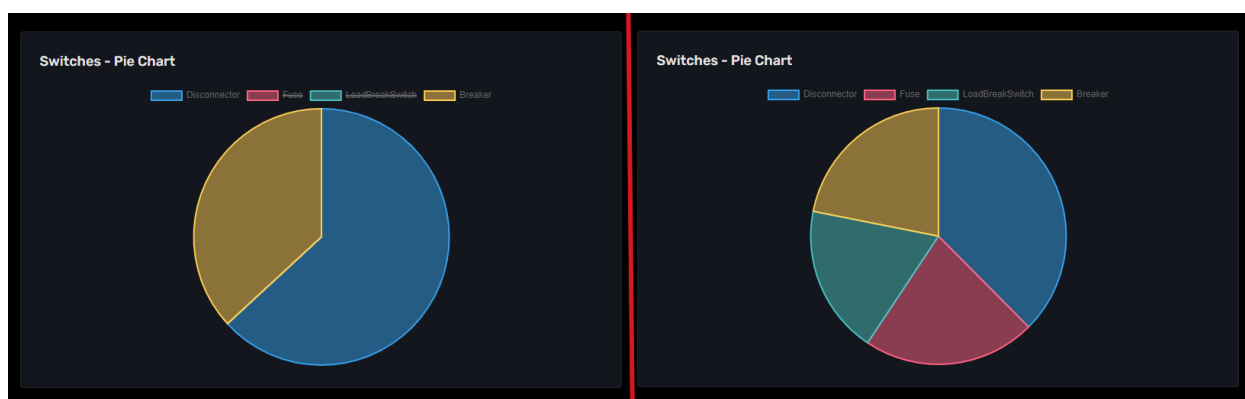
Други елемент јесу графови. Графички приказ података повећава квалитет и једноставност у приказу. На Dashboard станици постоје два графа. То су Substations и Switches графови.

Substations граф приказује бројно стање прекидача свих врста у сваком од трансформаторских станица. Ради боље прегледности, граф генерише наизменично изабрану боју којом попуњава део графа за новонасталу станицу. (Слика 17)

Switches представља графички приказ бројног стања исписан на блоковима са слике 16. Постоји и могућност отклањања неког елемента из графа ради креирања жељеног приказа. (Слика 18)



Слика 17 – Substations граф

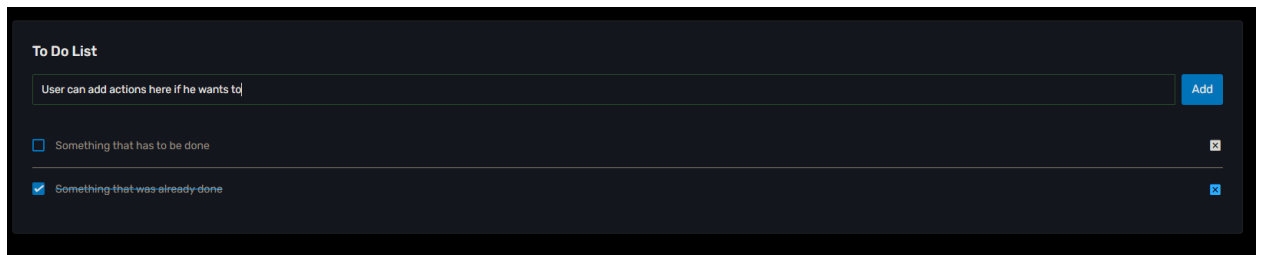


Слика 18 – Switches граф са два избачена прекидача и са свим прекидачима

Потребно је напоменути да је сваки прекидач обојен специфичном бојом која га обележава. Истим тим бојама обојени су и на графику. У каснијем поглављу о прегледу свих прекидача може се приметити да је тема самих страница попуњена том сппцифичном бојом.

- Disconnecter - Плава
- Fuse – Црвена
- LoadSwitchBreak - Зелена
- Breaker – Жута

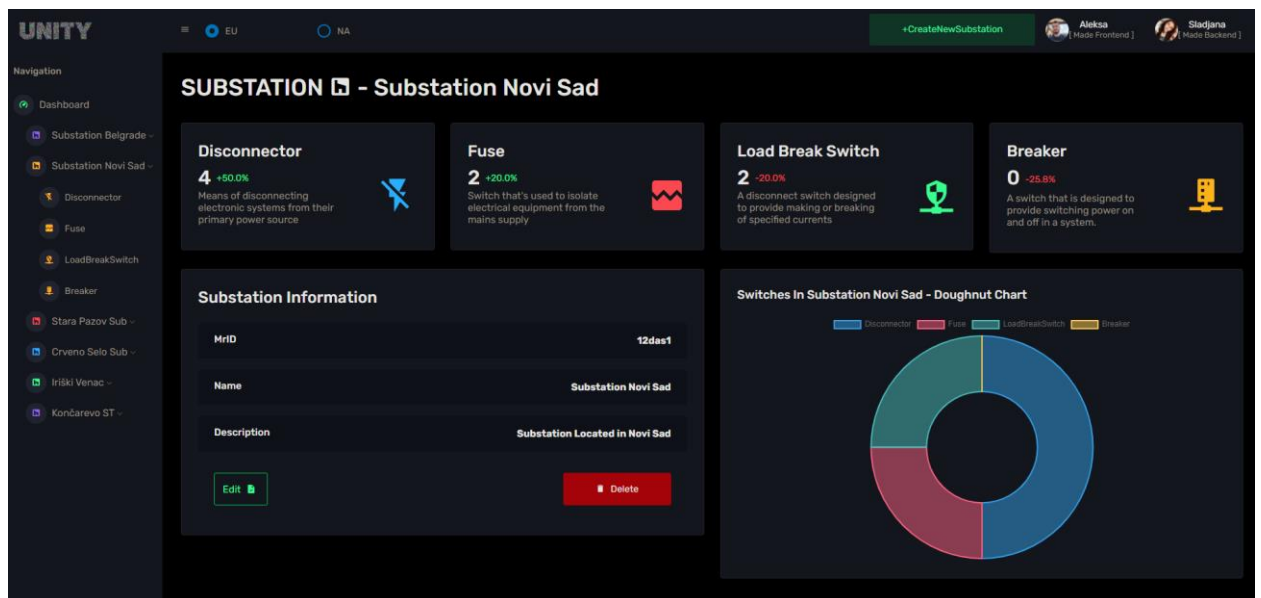
Трећи елемент Dashboard-а јесте „To-do“ листа. Њена улога је прилично једноставна. Додавање догађаја, њихово чекирање након извршавања и, ако корисник то жели, њихово уклањање.



Слика 19 - „To-do“ листа

7.5 Substation приказ

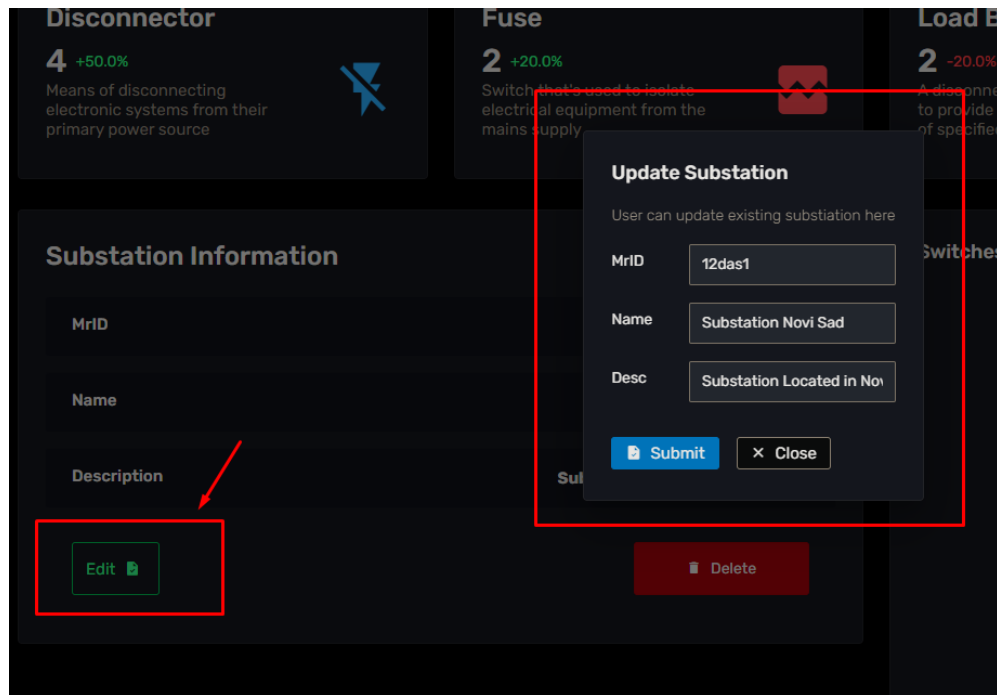
Кликом на сајдбар, на саму трансформаторску станицу, као што је већ речено, добићемо детаљнији приказ о њој. (Слика 20)



Слика 20 – Приказ изабране трансформаторске станице

Приказана станица такође се састоји од 3 елемента. Прва два елемента, блокови са информацијама и граф Switches идентични су као на Dashboard-у. Једина разлика је што показују информације са једне станице, а не са целог система. Блок информација нема више испис која трансформаторска станица има највише прекидача. Уместо тога садржи кратак опис сваког од њих. Такође тип графа више није Pie Chart већ Doughnut Chart ради естетских разлога.

Једини нови елемент јесте форма са исписаним детаљнијим информацијама о изабраној станици. Кликом на дугме „Edit“ корисник има могућност да измени дате са новим (валидним) информацијама. (Слика 21)



Слика 21 – Измена података трансформаторске станице

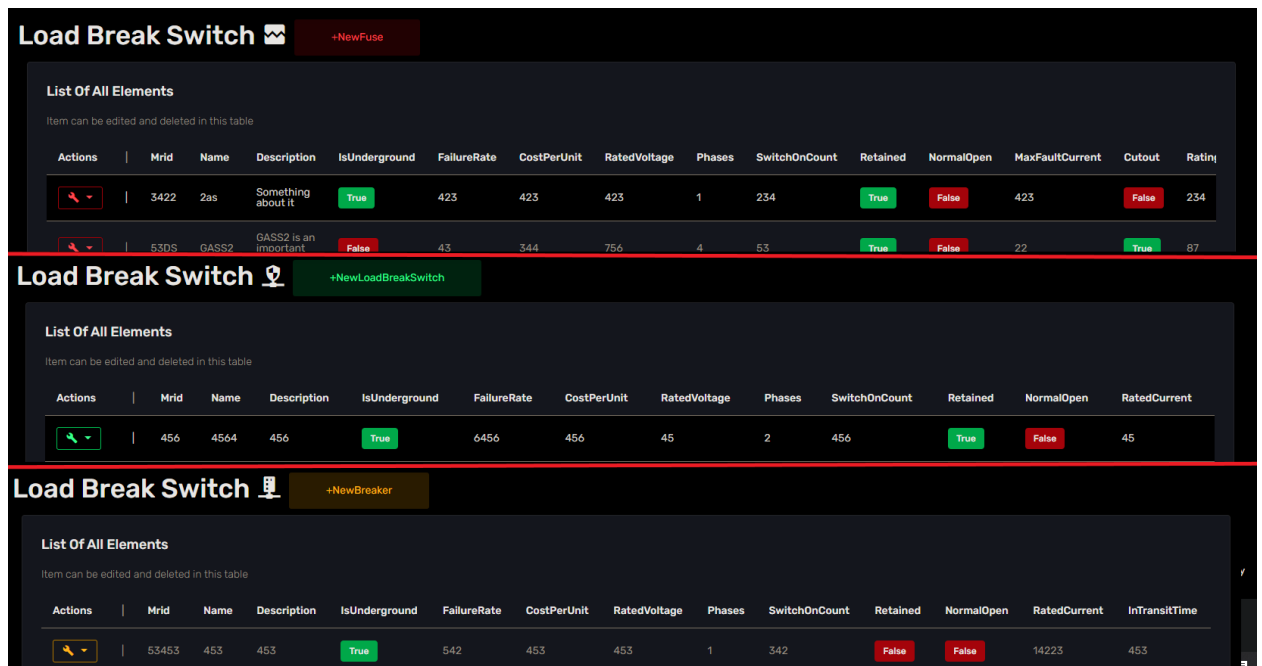
Такође кликом на дугме „Delete“ видљиво на слици 21, корисник може обрисати трансформаторску станицу као и све прекидаче који се налазе на њој. Овом акцијом та станица ће бити обрисана и са сајдбара.

7.6 Приказ листе појединачних прекидача

Након отварања детаљнијег приказа и падајућег менија за изабрану станицу, постоји могућност за приступање приказа листе за сваки појединачни прекидач. (Слика 22) Као што је већ наглашено свака од ових страница одговараће теми свог прекидача. (Слика 23)

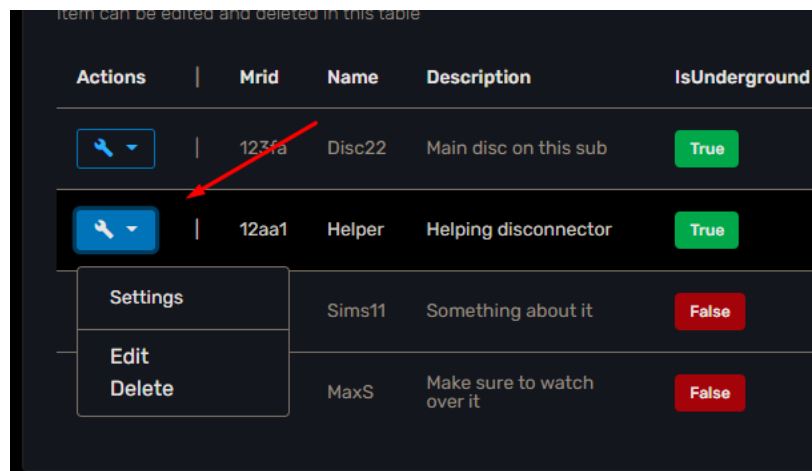
Load Break Switch  +NewDisconnector												
List Of All Elements												
Item can be edited and deleted in this table												
Actions	MrId	Name	Description	IsUnderground	FailureRate	CostPerUnit	RatedVoltage	Phases	SwitchOnCount	Retained	NormalOpen	ReactiveBreakingCurrent
	123fa	Disc22	Main disc on this sub	True	8	123	53	3	22	False	True	42
	12aa1	Helper	Helping disconnector	True	9	423	644	4	33	True	False	216
	34s2	Sims11	Something about it	False	1231	111	312	3	22	False	False	33
	423s	MaxS	Make sure to watch over it	False	31	311	566	1	33	True	True	122

Слика 22 – Приказ свих Disconnector-а у изабраној станици



Слика 23 – Приказ јединствене теме за сваки прекидач

Постоје операције које је могуће извести над елементима листе. Те операције добијају се кликом на поље Actions. Појавиће се падајући мени са опцијама „Edit“ и „Delete“. (Слика 24) Кликом на опцију „Delete“ елемент ће бити избрисан из листе (Све промене у систему које утичу на граф и остале елементе биће ажуриране у реалном времену).



Слика 24 – Падајући мени за елемент у листи

Кликом на опцију „Edit“ прозор са свим информацијама о елементу ће бити отворен. Корисник може у њему промени све информације. (Слика 25)

Кликом на дугме за додавање новог елемента (нпр. „NewDisconnecter“) који је лоциран поред наслова странице, отвориће се нови прозор, идентичан прозору за измене са празним пољима. (Слика 26)

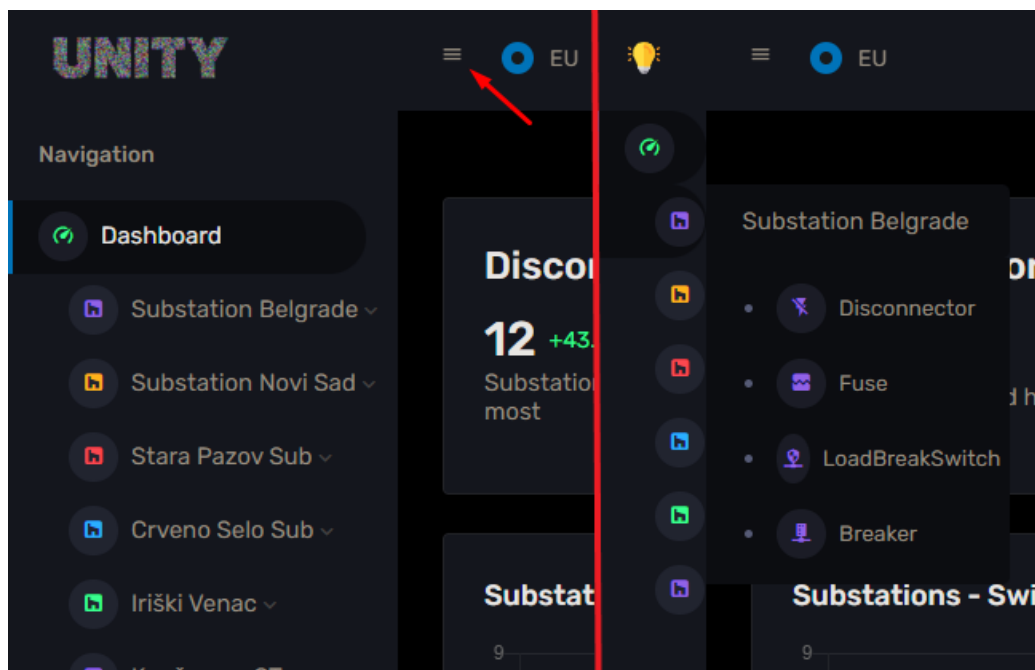
Слика 25 – Прозор за измену

Слика 26 – Прозор за додавање

7.7 Промена величине прозора и сајдбара

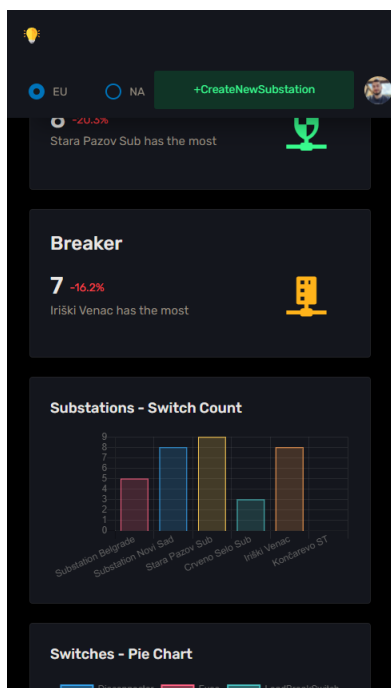
Апликација је реализована да буде флексибилна, уз могућност да се користи на разним уређајима. Уз промену прозора апликације, величина елемената ће се мењати по одговарајућој логици. Такође постоји и могућност увлачења и ширења сајдбара. Фокус на што бољи приказ елемената корисницима долази овде до изражаја.

Промена ширине сајдбара извршава се кликом да дугме на навбару. (Слика 27)

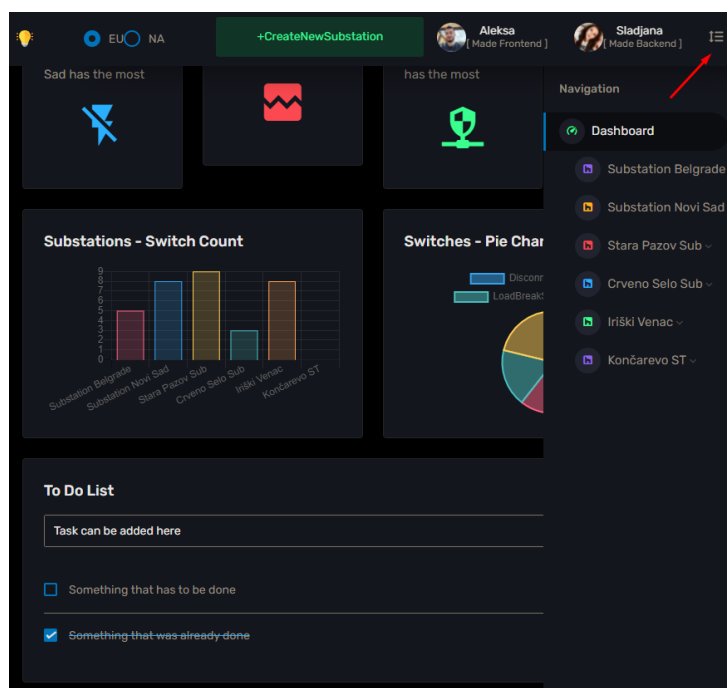


Слика 27 – Приказ промене димензија сајдбара

Апликација поседује изглед и за мобилне уређаје (Слика 28) као и за уређаје са средњом димензијом екрана (као што су таблети) (Слика 29). На следећој слици биће приказан пример изгледа апликације у тим димензијама.



Слика 28 – Мобилна ап.



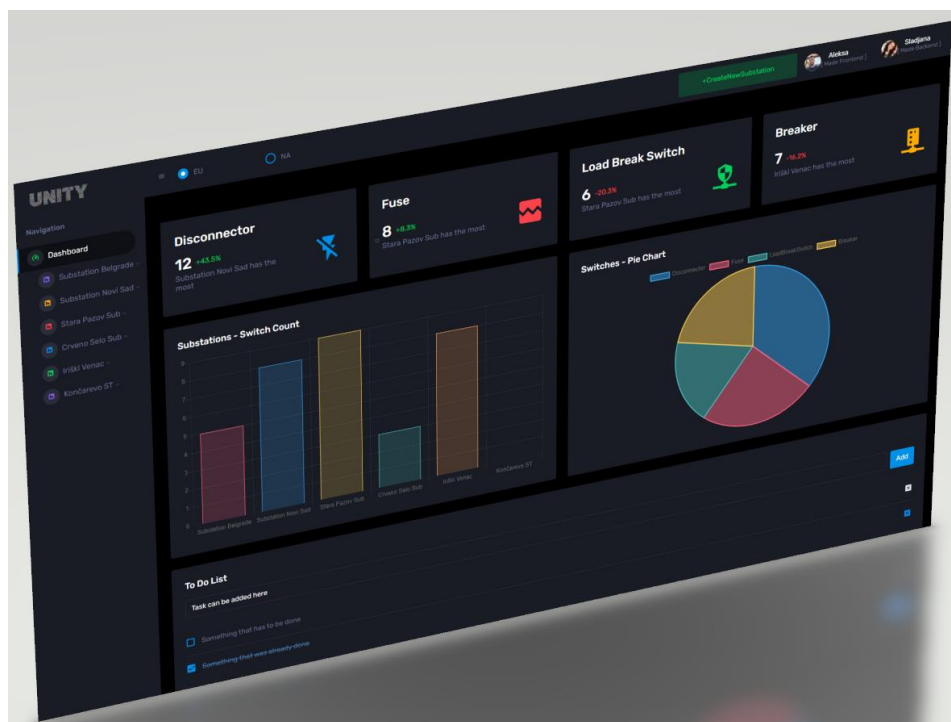
Слика 29 – таблет апликација

8. Закључак

Ова апликација представља предлог како би се веб технологије користиле за управљање електроенергетским системом. Прије свега фокус је на једноставности и прилагодљивости апликације за све кориснике, олакшавајући рад на њој. Применом разних графичких приказа, симбола и тема елемената испуњен је тражени задатак израђеног пројекта. Функционалност, рад у реалном времену и сам одзив апликације доведен је до високог нивоа, што је веома значајно за критичне системе попут електроенергетског. У оваквом систем оператеру је најважније да има правовремено ажуриране податке како би на најбољи могући начин управљао одређеним делом мреже.

Поред свих претходно поменутих особина, постоји велики простор за надоградњу ове апликације у области функционалности. Ова апликација омогућава преглед свих ентитета, њихово ажурирање, додавање и брисање. Међутим постоје многе додатне функционалности које могу унапредити рад са овом апликацијом. На пример могуће је имплементирати претрагу и филтрирање ентитета по различитим критеријумима, као и њихово сортирање. Такође могуће је реализовати преглед свих трансформаторских станица и прекидача на одређеној мапи. Омогућити кориснику да кликом на њих добију све потребне информације и да сами ентитети буду представљени на датој мапи симболима који су реализовани у овој апликацији. Додавањем одређених нотификација у систему и могућности комуникације између оператера, у случају кварова или битних измена такође би веома унапредили саму ефикасност дате апликације.

Једина мана ове апликације и реализације електроенергетског система на веб серверу, јесте то што је потребно обезбедити сталну конекцију са интернетом. У супротном може доћи до отказа система или неког другог квара, без да сами оператери буду обавештени о томе.



Слика 30 – Израђена апликација

9. Literatura

- [1] Martin Fowler: "Patterns of Enterprise Application Architecture", 2002.
- [2] Alex Banks : "Learning React: Functional Web Development with React and Redux"
- [3] Leonard Richardson: "RESTful Web APIs: Services for a Changing World"
- [4] Ilya Grigorik: "High-Performance Browser Networking"
- [5] "All things React" : <https://www.telerik.com/blogs/all-things-react> (septembar 2021)

Биографија

Алекса Попов рођен је 05. фебруара 1998. године у Зрењанину. Завршио је основну школу „Бошко Павковљевић Пинки“ у Старој Пазови, а затим општи смер гимназије у оквиру средњошколског центра „Бранко Радичевић“ у Старој Пазови. Школске 2017. године уписује Факултет техничких наука у Новом Саду, на студијском програму Примењено Софтверско Инжењерство. Положио је све испите прописане планом и програмом.